



Single Perceptron AI Hardware Accelerator

MARIAM HOSSAM MOHAMED - OMAR SHERIF ZAKI
AMR KHALED MOHAMED - ROAA SHERIFF SAYED
OMAR MOHAMED SALEM

1. Introduction:.....	2
2. System Architecture & Specifications:	2
3. Hardware Datapath & Modules.....	2
4. Design flow:	3
5. Top-Level Interface (I/O):	3
6. RTL code:.....	4
- Mac unit:	4
- Fsm:	5
- Bias add:	7
- ReLU:.....	7
- Top module:	8
7. Challenges & solution:.....	9
- Challenge 1: Long Critical Path (Timing Violations)	10
- Challenge 2: Scaling to a Synchronized Neural Network Layer	11
8. Test and Simulation	12
- Simulation of MAC unit:.....	13
- Simulation of FSM:	14
9. Testbench and Simulation of perceptron_top	15
- Testbench code:	15
Test Case 1: Standard Positive Inputs	19
Test Case 2: Asynchronous Reset Recovery Test	19
Test Case 3: Standard Negative Inputs	19
Test Case 4: Mixed Extreme Bounds Multiplication	19
Test Case 5: Full-Scale Accumulator Stress Test.....	20
Test Case 6: Non-Zero Bias Addition	20
- Simulation Results:	20
10. Waveform:.....	21
- Test case 1	21
- Test case 2	22
- Test case 3	22

- Test case 4	22
- Test case 5	23
- Test case 6	23
11. Schematics:.....	25
- Perceptron schematic	25
- Layer schematic	25

1. Introduction:

This project proposes the RTL design of a Single Perceptron AI Hardware Accelerator. The perceptron is the fundamental building block of Deep Neural Networks (DNNs), translating biological neuron mechanisms into a digital hardware data path.



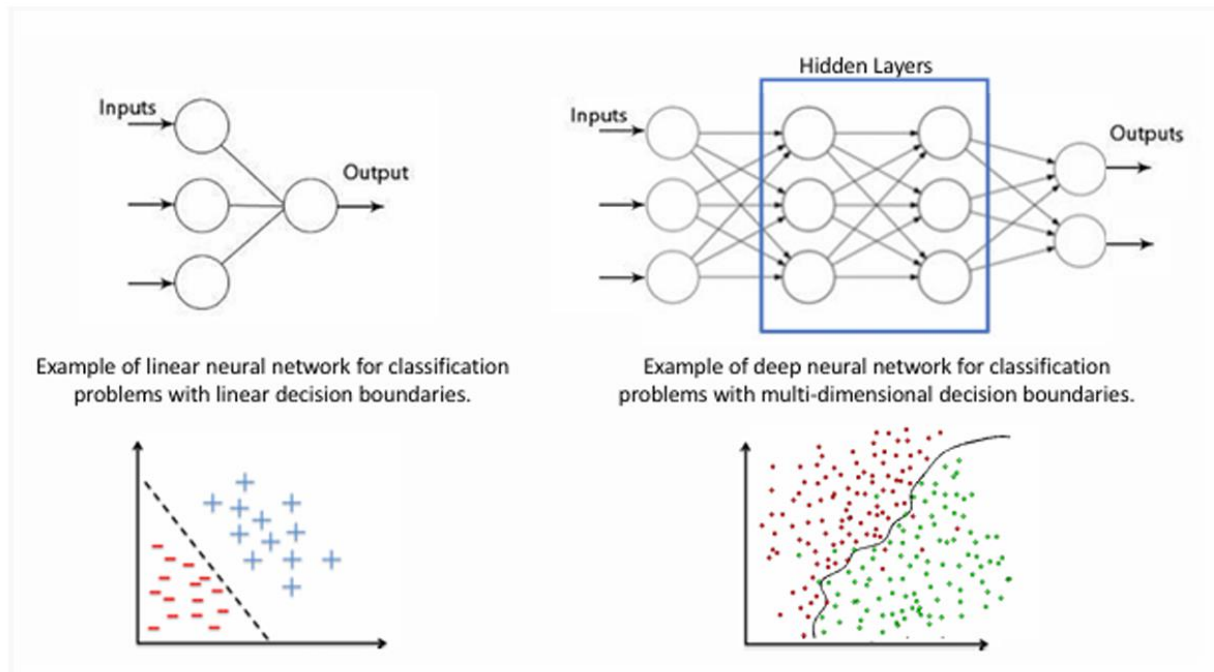
2. System Architecture & Specifications:

To optimize embedded hardware constraints and real-time streaming, the design uses a Feed-Forward Datapath parameterized for different bit-widths. The default parameterization is set as follows:

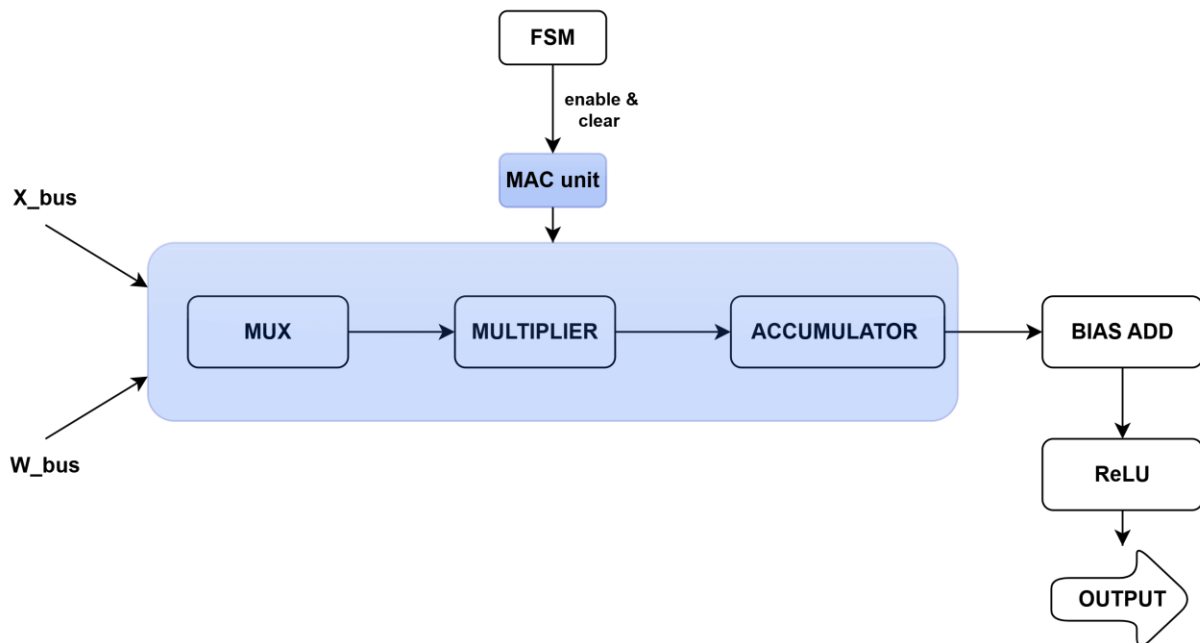
- Parameter DATA_WIDTH = 8; Minimizes area and latency by using 8-bit signed integers for inputs and weights instead of floating-point.
- Parameter ACC_WIDTH = 32; Prevents data overflow during the Multiply-Accumulate (MAC) process.
- ReLU Activation: Replaces complex sigmoids with a simple MSB-check logic.

3. Hardware Datapath & Modules

The system is divided into parallel-developed modules. The data path flows sequentially from the external inputs to the final decision.



4. Design flow:



5. Top-Level Interface (I/O):

The external interface accepts streaming data to reflect real-world AI applications where inputs (e.g., sensor data or image pixels) change dynamically.

Port Name	Width	Direction	Description
clk	1	input	System clock.
rst_n	1	input	Active-low asynchronous reset.
start	1	input	Trigger to begin computation.
data_valid	1	input	Indicates the input data on the bus is valid.
x_in	[DATA_WIDTH-1:0]	input	Streaming feature input (e.g., 8-bit).
w_in	[DATA_WIDTH-1:0]	input	Streaming weight input (e.g., 8-bit).
final_output	[ACC_WIDTH-1:0]	output	Decision value after ReLU (e.g., 32-bit).
done	1	output	Flag indicating that computation is complete.

6. RTL code:

- Mac unit:

The MAC (Multiply-Accumulate) unit calculates the weighted sum for the perceptron. It takes a streaming input feature(x_{in}), multiplies it by its assigned weight(w_{in}), and adds that product to a running total, which is then passed to the bias and activation stages.

Structural Decomposition:

- **Combinational Multiplier:** Constantly calculates the product of x_{in} and w_{in} .
- **Sequential Accumulator:** An adder circuit wired to a 32-bit clocked register that safely stores the running total on every positive clock edge

```

> mariam > NTI > project > mac_unit(1).v
1  module mac_unit #
2  (
3      parameter DATA_WIDTH = 8,
4      parameter ACC_WIDTH = 32
5  )
6  (
7      // in Ports
8      input wire signed [(DATA_WIDTH - 1):0] x_in,
9      input wire signed [(DATA_WIDTH - 1):0] w_in,
10     input wire clk,
11     input wire rst_n,
12     input wire mac_en,
13     input wire mac_clear,
14
15     // Output port
16     output reg signed [(ACC_WIDTH - 1):0] mac_out
17 );
18
19     always @(posedge clk or negedge rst_n) begin
20         if (!rst_n) begin
21             mac_out <= {ACC_WIDTH{1'b0}};
22         end
23         else if (mac_clear) begin
24             mac_out <= {ACC_WIDTH{1'b0}};
25         end
26         else if (mac_en) begin
27             mac_out <= mac_out + (x_in * w_in);
28         end
29     end
30
31 endmodule

```

- Fsm:

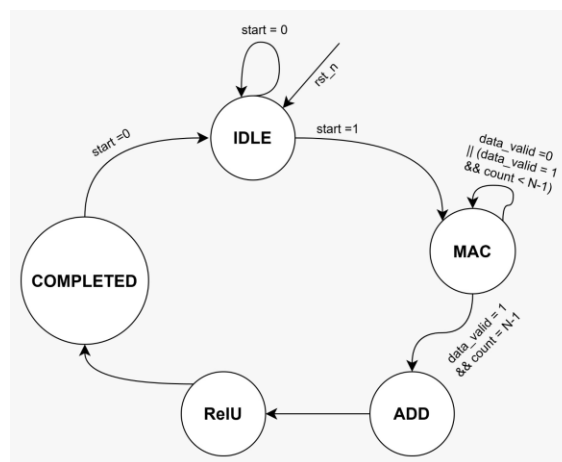
The FSM (Finite State Machine) coordinates the data path sequentially, deciding exactly when the system should wait for data, accumulate inputs, add the bias, apply the ReLU activation, and flag the final result as ready.

It contains a counter which tracks exactly how many valid input features have been processed. It increments only during the MAC state when `data_valid` is high. Once it reaches the target number (N-1), it triggers the FSM to exit the accumulation phase and move to the ADD state, ensuring the exact required number of inputs are gathered before finalizing the math.

Outputs:

- **mac_clear**: Asserted during the IDLE state to flush the accumulator to zero, guaranteeing a clean slate before a new calculation begins.
- **mac_en**: Asserted during the MAC state (only when data_valid is high) to allow accumulation. It acts as a gatekeeper, preventing the MAC unit from adding garbage data or calculating during the bias and ReLU phases.
- **done flag**: Asserted when the FSM enters the COMPLETED state. It acts as a signal to external systems that the entire perceptron calculation is finished, and the stable, final result is ready to be read from the output bus.

State diagram:



```
> mariam > NTI > project > fsm_controller.v
1  module fsm_controller (clk,rst_n,start,data_valid,done,mac_en,clear);
2      localparam IDLE = 3'd0 ;
3      localparam MAC = 3'd1 ;
4      localparam ADD = 3'd2 ;
5      localparam ReLU = 3'd3 ;
6      localparam COMPLETED = 3'd4 ;
7      parameter N = 8; // number of inputs
8      input start,data_valid,clk,rst_n;
9      output done,mac_en,clear;
10     reg [2:0] cs,ns;
11     reg [$clog2(N)-1:0] count;
12
13     always @(posedge clk or negedge rst_n) begin
14         if (~rst_n) cs <= IDLE;
15         else cs <= ns;
16     end
17
18     always @(posedge clk or negedge rst_n) begin
19         if (~rst_n)
20             count <= 'd0;
21         else if (cs == IDLE || cs == COMPLETED) count <= 'd0;
22         else if (cs == MAC && data_valid == 1'b1) count <= count + 1'b1;
23     end
```

```

25     always @(*) begin
26         ns = cs;
27         case (cs)
28             IDLE:
29                 if (start == 1'b1) ns = MAC;
30             MAC:
31                 if (data_valid == 1'b1 && count == N-1) ns = ADD;
32             ADD:
33                 ns = ReLU;
34             ReLU:
35                 ns = COMPLETED;
36             COMPLETED:
37                 if (start == 0) ns = IDLE;
38             default:
39                 ns = IDLE;
40         endcase
41     end
42     assign done = (cs == COMPLETED);
43     assign mac_en = (cs == MAC && data_valid == 1'b1); // tell mac to accumulate only on valid data
44     assign clear = (cs == IDLE); // hold mac unit while waiting
45 endmodule

```

- Bias add:

The bias unit shifts the final accumulated sum by a set amount before it hits the activation function. In neural networks, adding this bias offset is essential because it allows the perceptron to learn patterns that do not naturally pass through the zero-origin point.

Structural Decomposition:

- **Sign Extender:** A structural wiring operation. It requires no actual logic gates; the synthesizer just routes the 7th bit of the bias to the top 24 wires of the internal bus.
- **Combinational Adder:** A standard 32-bit hardware adder circuit that instantly computes the sum without waiting for a clock edge or an FSM enable signal.

```

> mariam > NTI > project > Bias_Add_Unit.v
1  module Bias_Add_Unit#(parameter DATA_WIDTH = 8,parameter ACC_WIDTH = 32)
2  ( input wire signed [DATA_WIDTH-1:0] bias,
3    input wire signed [ACC_WIDTH-1:0] acc,
4    output wire signed [ACC_WIDTH-1:0] final_sum
5  );
6  wire signed [ACC_WIDTH-1:0] bias_32;
7  assign bias_32 = {(ACC_WIDTH-DATA_WIDTH){bias[DATA_WIDTH-1]}},bias} ;
8  assign final_sum = acc + bias_32 ;
9  endmodule

```

- ReLU:

The ReLU (Rectified Linear Unit) serves as the activation function for the perceptron. It introduces non-linearity into the neural network's decision-making process by passing positive totals through unaltered while instantly clamping any negative totals to zero. output is 0 when MSB is 1 (negative value).

```
> mariam > NTI > project > ReLU (1).v
1  module ReLU#(parameter ACC_WIDTH = 32)(
2      input wire signed [ACC_WIDTH-1:0] final_sum,
3      output wire signed [ACC_WIDTH-1:0] ReLU_out
4  );
5
6  assign ReLU_out=(final_sum[ACC_WIDTH-1]==1)? 0 : final_sum ;
7
8  endmodule
```

- Top module:

The top module (perceptron_top) acts as the structural wrapper for the entire AI accelerator. Its purpose is to integrate the individual sub-modules (FSM, MAC, Bias, and ReLU) into a single, unified system and connect them to the outside world via the external input/output pins.

```
> mariam > NTI > project > perceptron_top.v
1  module perceptron_top
2      #(
3      parameter DATA_WIDTH = 8,
4      parameter ACC_WIDTH = 32,
5      parameter N = 8
6      )
7      (
8      input wire          clk,
9      input wire          rst_n,
10     input wire          start,
11     input wire          data_valid,
12     input wire signed [DATA_WIDTH-1 : 0] x_in,
13     input wire signed [DATA_WIDTH-1 : 0] w_in,
14     input wire signed [DATA_WIDTH-1 : 0] bias,
15
16
17     output wire signed [ACC_WIDTH-1 : 0] final_output,
18     output wire          done
19 );
20
21 wire mac_en_sig;
22 wire mac_clear_sig;
23 wire signed [ACC_WIDTH-1 : 0] mac_to_bias_sum;
24 wire signed [ACC_WIDTH-1 : 0] bias_to_relu_sum;
```

```

26
27     fsm_controller
28     #(.N(N))
29     top_fsm
30     (
31         .clk(clk),
32         .rst_n(rst_n),
33         .start(start),
34         .data_valid(data_valid),
35         .done(done),
36         .mac_en(mac_en_sig),
37         .clear(mac_clear_sig)
38     );
39
40     mac_unit
41     #(
42         .DATA_WIDTH(DATA_WIDTH),
43         .ACC_WIDTH(ACC_WIDTH)
44     )
45     top_mac_unit
46     (
47         .x_in(x_in),
48         .w_in(w_in),
49         .clk(clk),
50         .rst_n(rst_n),
51         .mac_en(mac_en_sig),
52         .mac_clear(mac_clear_sig),
53         .mac_out(mac_to_bias_sum)
54     );

```

```

56
57     Bias_Add_Unit
58     #(
59         .DATA_WIDTH(DATA_WIDTH),
60         .ACC_WIDTH(ACC_WIDTH)
61     )
62     top_Bias_Add_Unit
63     (
64         .bias(bias),
65         .acc(mac_to_bias_sum),
66         .final_sum(bias_to_relu_sum)
67     );
68
69
70     ReLU
71     #(.ACC_WIDTH(ACC_WIDTH))
72     top_ReLU
73     (
74         .final_sum(bias_to_relu_sum),
75         .ReLU_out(final_output)
76     );
77
78     endmodule
79

```

7. Challenges & solution:

- Challenge 1: Long Critical Path (Timing Violations)

In the original MAC unit, the hardware multiplier and the accumulator (adder) were chained together in the same clock cycle. Multipliers are complex, slow hardware blocks. Forcing the signal to propagate through a multiplier and immediately through an adder creates a long combinational "critical path." This limits the maximum clock frequency (fmax).

Using Pipelining as a solution:

- By inserting a register (mult_pipe_reg) directly after the multiplier to break the critical path into two shorter stages.
- Stage 1 now calculates $x_{in} * w_{in}$ and locks it in memory.
- Stage 2 takes that locked value, explicitly sign-extends it, and adds it to mac_out.

To ensure the FSM control stays perfectly synced with this new 1-cycle data delay, pipelining the enable signal using a D-Flip-Flop ($mac_en_d \leq mac_en$).

```
> mariam > NTI > project > mac_unit (2).v
1  module mac_unit #
2  (
3      parameter DATA_WIDTH = 8,
4      parameter ACC_WIDTH = 32
5  )
6  (
7      // in Ports
8      input wire signed [(DATA_WIDTH - 1):0] x_in,
9      input wire signed [(DATA_WIDTH - 1):0] w_in,
10     input wire clk,
11     input wire rst_n,
12     input wire mac_en,
13     input wire mac_clear,
14
15     // Output port
16     output reg signed [(ACC_WIDTH - 1):0] mac_out
17 );
18 reg signed [(2*DATA_WIDTH)-1 : 0] mult_pipe_reg;
19 reg mac_en_d;
```

```

21 always @(posedge clk or negedge rst_n) begin
22     if (!rst_n) begin
23         mult_pipe_reg <= 16'sd0;
24         mac_en_d <= 1'b0;
25         mac_out <= {ACC_WIDTH{1'b0}};
26     end
27     else if (mac_clear) begin
28         mult_pipe_reg <= 16'sd0;
29         mac_en_d <= 1'b0;
30         mac_out <= {ACC_WIDTH{1'b0}};
31     end
32     else begin
33         mult_pipe_reg <= x_in * w_in;
34         mac_en_d <= mac_en;
35         if (mac_en_d) begin
36             mac_out <= mac_out + $signed({(ACC_WIDTH - 2*DATA_WIDTH){mult_pipe_reg[(2*DATA_WIDTH)-1]}}, mult_pipe_reg));
37         end
38     end
39 end
40
41 endmodule

```

- Challenge 2: Scaling to a Synchronized Neural Network Layer

When upgrading from one single perceptron to a whole group of them working together, two big issues popped up:

1. **Wiring:** Verilog struggles with sending multi-dimensional arrays (like a grid of wires). We needed a way to cleanly send specific weights and biases to each individual neuron.
2. **Timing:** All neurons do their math at the exact same time. The main system needs one single "finished" signal for the whole group, rather than checking every single neuron one by one.

Using perceptron layer as solution:

- Instead of trying to send a complex grid of wires, packing all the weights and biases into one super-long, flat wire (like `w_in_flat`). Then, using a generate for loop to automatically build neurons. Inside that loop, using Verilog's slicing tool (`+:`) to chop that long wire into perfectly sized chunks, feeding each neuron exactly the data it needed.
- To know when the whole layer is done, wiring every neuron's individual done signal into one shared bus (`done_bus`). Then, using a simple logic trick called a Reduction AND (`&done_bus`) to only raise the final `layer_done` flag when every single neuron has raised its hand to say it is finished.

```

> mariam > NTI > project > ≡ perceptron_layer.v
1  `timescale 1ns / 1ps
2  module perceptron_layer
3  #(
4      parameter DATA_WIDTH = 8,
5      parameter ACC_WIDTH = 32,
6      parameter N = 8,
7      parameter NUM_NEURONS = 4
8  )
9  (
10     input wire          clk,
11     input wire          rst_n,
12     input wire          start,
13     input wire          data_valid,
14     input wire signed [DATA_WIDTH-1 : 0] x_in,
15
16     input wire signed [(NUM_NEURONS*DATA_WIDTH)-1:0] w_in_flat,
17     input wire signed [(NUM_NEURONS*DATA_WIDTH)-1:0] bias_flat,
18
19     output wire signed [(NUM_NEURONS*ACC_WIDTH)-1:0] out_flat,
20     output wire          layer_done
21 );
22
23     wire [NUM_NEURONS-1 : 0] done_bus;
24     genvar i;

```

```

27     generate
28         for(i=0 ; i < NUM_NEURONS ; i = i + 1)
29             begin : perceptron_array
30                 perceptron_top
31                 #(
32                     .DATA_WIDTH(DATA_WIDTH),
33                     .ACC_WIDTH(ACC_WIDTH),
34                     .N(N)
35                 )
36                 u_perceptron
37                 (
38                     .clk(clk),
39                     .rst_n(rst_n),
40                     .start(start),
41                     .data_valid(data_valid),
42                     .x_in(x_in),
43
44                     .w_in(w_in_flat[i*DATA_WIDTH +: DATA_WIDTH]),
45                     .bias(bias_flat[i*DATA_WIDTH +: DATA_WIDTH]),
46                     .final_output(out_flat[i*ACC_WIDTH +: ACC_WIDTH]),
47                     .done(done_bus[i])
48                 );
49             end
50         endgenerate
51
52         assign layer_done = &done_bus;
53
54     endmodule
55

```

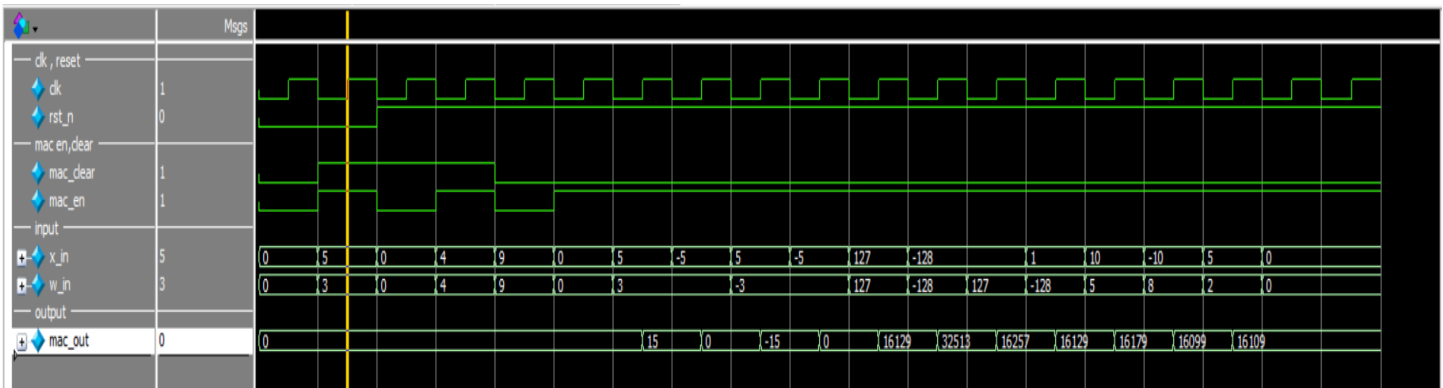
8. Test and Simulation

The two primary functional blocks within the RTL architecture are the MAC (Multiply-Accumulate) Unit and the FSM (Finite State Machine) Controller.

- The MAC Unit is responsible for multiplying signed 8-bit data features by signed 8-bit weights. It then accumulates these products with the previous output value (accumulating sum of products), resulting in a signed 16-bit output (mac_out) that represents the total accumulated sum of the input products.

- Simulation of MAC unit:

```
Transcript
# Time = 0|x = 0|w = 0|output = 0
# Time = 100|x = 5|w = 3|output = 0
# Time = 200|x = 0|w = 0|output = 0
# Time = 300|x = 4|w = 4|output = 0
# Time = 400|x = 9|w = 9|output = 0
# Time = 500|x = 0|w = 0|output = 0
# Time = 600|x = 5|w = 3|output = 0
# Time = 650|x = 5|w = 3|output = 15
# Time = 700|x = -5|w = 3|output = 15
# Time = 750|x = -5|w = 3|output = 0
# Time = 800|x = 5|w = -3|output = 0
# Time = 850|x = 5|w = -3|output = -15
# Time = 900|x = -5|w = -3|output = -15
# Time = 950|x = -5|w = -3|output = 0
# Time = 1000|x = 127|w = 127|output = 0
# Time = 1050|x = 127|w = 127|output = 16129
# Time = 1100|x = -128|w = -128|output = 16129
# Time = 1150|x = -128|w = -128|output = 32513
# Time = 1200|x = -128|w = 127|output = 32513
# Time = 1250|x = -128|w = 127|output = 16257
# Time = 1300|x = 1|w = -128|output = 16257
# Time = 1350|x = 1|w = -128|output = 16129
# Time = 1400|x = 10|w = 5|output = 16129
# Time = 1450|x = 10|w = 5|output = 16179
# Time = 1500|x = -10|w = 8|output = 16179
# Time = 1550|x = -10|w = 8|output = 16099
# Time = 1600|x = 5|w = 2|output = 16099
# Time = 1650|x = 5|w = 2|output = 16109
# Time = 1700|x = 0|w = 0|output = 16109
# ** Note: $stop : M:/NTII Digital Design Using FPGA/Final Project/Mac_Unit/tb_mac_unit.v(91)
# Time: 190 ns Iteration: 0 Instance: /tb_mac_unit
```



First positive clock edge: $\text{rst_n} = 0$ (active-low reset), $\text{mac_en} = 0$, $\text{mac_clear} = 0$, $\text{x_in} = 0$, and $\text{w_in} = 0$. Due to the asynchronous reset, mac_out is forced to 0.

Second positive clock edge: $\text{rst_n} = 0$ (active-low reset), $\text{mac_en} = 1$, $\text{mac_clear} = 1$, $\text{x_in} = 5$, and $\text{w_in} = 3$. The reset condition overrides both the clear and enable signals, maintaining mac_out at 0.

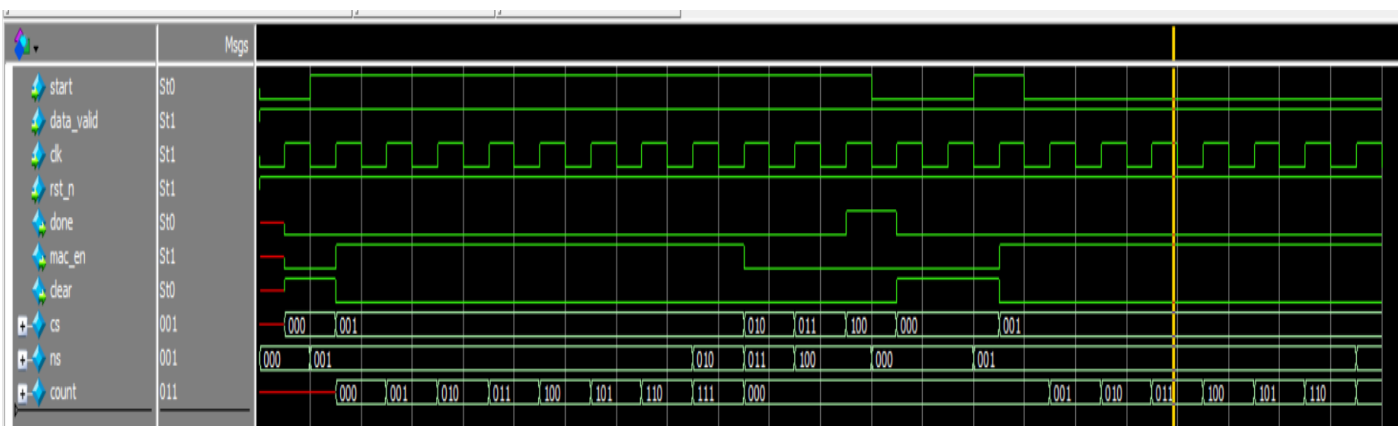
Third positive clock edge: $\text{rst_n} = 1$ (inactive reset), $\text{mac_en} = 0$, $\text{mac_clear} = 1$, $\text{x_in} = 0$, and $\text{w_in} = 0$. The synchronous clear is triggered, ensuring mac_out remains 0.

Fourth positive clock edge: $\text{rst_n} = 1$, $\text{mac_en} = 1$, $\text{mac_clear} = 1$, $\text{x_in} = 4$, and $\text{w_in} = 4$. The clear signal takes priority over the enable signal, so mac_out remains 0.

Following the successful verification of rst_n , mac_en , and mac_clear , the complete functionality of the MAC unit is confirmed through the testbench output logs and the resulting mac_out values.

- The FSM Controller coordinates the sequential execution of all operational stages. It manages when the MAC unit receives input sample pairs, when the bias term is added, when the value is passed to the ReLU activation function, and when the entire computation sequence concludes.

- Simulation of FSM:



Having validated the MAC unit, the verification process transitions to the functional simulation and testbench execution of the top-level module.

9. Testbench and Simulation of perceptron_top

- Testbench code:

```

1  `timescale 1ns/100ps
2
3  module tb_perceptron_top;
4
5      parameter clkPeriod = 10;
6      parameter DATA_WIDTH = 8;
7      parameter ACC_WIDTH = 32;
8      parameter N = 8;
9
10     reg clk;
11     reg rst_n;
12     reg start;
13     reg data_valid;
14     reg signed [DATA_WIDTH-1:0] x_in;
15     reg signed [DATA_WIDTH-1:0] w_in;
16     reg signed [DATA_WIDTH-1:0] bias;
17
18     wire signed [ACC_WIDTH-1:0] final_output;
19     wire done;
20
21     perceptron_top #(
22         .DATA_WIDTH(DATA_WIDTH),
23         .ACC_WIDTH(ACC_WIDTH),
24         .N(N)
25     ) DUT (
26         .clk          (clk),
27         .rst_n        (rst_n),
28         .start        (start),
29         .data_valid   (data_valid),
30         .x_in         (x_in),
31         .w_in         (w_in),
32         .bias         (bias),
33         .final_output (final_output),
34         .done         (done)
35     );
36
37     always #(clkPeriod/2) clk = ~clk;
38
39     always @(posedge done) begin
40         $display("[t=%0t ps] Internal Raw Sum = %0d | final_output = %0d", $time, $signed(DUT.top_mac_unit.mac_out), final_output);
41     end
42
43     initial
44     begin
45         $monitor("Time = %t|rst_n = %b|start = %b|data_valid = %b|x_in = %d|w_in = %d|bias = %d|done = %b|final_output = %d", $time,
46             rst_n, start, data_valid, x_in, w_in, bias, done, final_output);
47         clk = 0;      rst_n = 0;
48         start = 0;    data_valid = 0;
49         x_in = 8'sd1; w_in = 8'sd1;
50         bias = 0;
51
52         #(clkPeriod*2);
53         rst_n = 1;
54         x_in = 0; w_in = 0;
55
56         //1st 8 input pairs
57         #(clkPeriod); start = 1; // 3shan e1 idle => mac
58         #(clkPeriod); data_valid = 1;
59         x_in = 8'sd5; w_in = 8'sd4;
60         #(clkPeriod); start = 0;
61         x_in = 8'sd3; w_in = 8'sd6;
62         #(clkPeriod);
63         x_in = 8'sd2; w_in = 8'sd1;
64         #(clkPeriod);
65         x_in = 8'sd4; w_in = 8'sd2;
66         #(clkPeriod);
67         x_in = 8'sd1; w_in = 8'sd5;
68         #(clkPeriod);
69         x_in = 8'sd6; w_in = 8'sd1;
70         #(clkPeriod);
71         x_in = 8'sd2; w_in = 8'sd3;
72         #(clkPeriod);
73         x_in = 8'sd3; w_in = 8'sd2;
74         wait(done); data_valid <= 0;
75     end

```

```

76
77 //test for reset
78 #(clkPeriod); rst_n = 0;
79 #(clkPeriod); rst_n = 1;
80
81
82 //2nd 8 pairs
83 #(clkPeriod); start = 1;
84 #(clkPeriod); data_valid = 1;
85 x_in = -8'sd10; w_in = 8'sd3;
86 #(clkPeriod); start = 0;
87 start = 0;
88 x_in = -8'sd5; w_in = 8'sd2;
89 #(clkPeriod);
90 x_in = -8'sd3; w_in = 8'sd4;
91 #(clkPeriod);
92 x_in = -8'sd1; w_in = 8'sd1;
93 #(clkPeriod);
94 x_in = -8'sd2; w_in = 8'sd5;
95 #(clkPeriod);
96 x_in = -8'sd4; w_in = 8'sd2;
97 #(clkPeriod);
98 x_in = -8'sd1; w_in = 8'sd3;
99 #(clkPeriod);
100 x_in = -8'sd2; w_in = 8'sd1;
101 wait(done); data_valid <= 0;

```

```

105 //neg max *
106 #(clkPeriod*2); start = 1;
107 #(clkPeriod); data_valid = 1;
108 x_in = -8'sd128; w_in = -8'sd128;
109 #(clkPeriod); start = 0;
110 x_in = 8'sd0; w_in = 8'sd0;
111 #(clkPeriod);
112 x_in = 8'sd0; w_in = 8'sd0;
113 #(clkPeriod);
114 x_in = 8'sd0; w_in = 8'sd0;
115 #(clkPeriod);
116 x_in = 8'sd0; w_in = 8'sd0;
117 #(clkPeriod);
118 x_in = 8'sd0; w_in = 8'sd0;
119 #(clkPeriod);
120 x_in = 8'sd0; w_in = 8'sd0;
121 #(clkPeriod);
122 x_in = 8'sd0; w_in = 8'sd0;
123 wait(done); data_valid <= 0;
124
125
126 // neg max*pos max
127 #(clkPeriod); start = 1;
128 #(clkPeriod); data_valid = 1;
129 x_in = -8'sd128; w_in = 8'sd127;
130 #(clkPeriod); start = 0;
131 x_in = 8'sd0; w_in = 8'sd0;
132 #(clkPeriod);
133 x_in = 8'sd0; w_in = 8'sd0;
134 #(clkPeriod);
135 x_in = 8'sd0; w_in = 8'sd0;
136 #(clkPeriod);
137 x_in = 8'sd0; w_in = 8'sd0;
138 #(clkPeriod);
139 x_in = 8'sd0; w_in = 8'sd0;
140 #(clkPeriod);
141 x_in = 8'sd0; w_in = 8'sd0;
142 #(clkPeriod);
143 x_in = 8'sd0; w_in = 8'sd0;
144 wait(done); data_valid <= 0;
145
146

```

```

147
148 //all max *
149 #(clkPeriod*2); start = 1;
150 #(clkPeriod); data_valid = 1;
151 x_in = 8'sd127; w_in = 8'sd127;
152 #(clkPeriod); start = 0;
153 x_in = 8'sd127; w_in = 8'sd127;
154 #(clkPeriod);
155 x_in = 8'sd127; w_in = 8'sd127;
156 #(clkPeriod);
157 x_in = 8'sd127; w_in = 8'sd127;
158 #(clkPeriod);
159 x_in = 8'sd127; w_in = 8'sd127;
160 #(clkPeriod);
161 x_in = 8'sd127; w_in = 8'sd127;
162 #(clkPeriod);
163 x_in = 8'sd127; w_in = 8'sd127;
164 #(clkPeriod);
165 x_in = 8'sd127; w_in = 8'sd127;
166 wait(done); data_valid <= 0;

```

```

167
168
169
170     #(clkPeriod); bias = -128;
171     #(clkPeriod); start = 1;
172     #(clkPeriod); data_valid = 1;
173     x_in = 8'sd5;   w_in      = 8'sd2;
174     #(clkPeriod); start = 0;
175     x_in = 8'sd5;   w_in      = 8'sd2;
176     #(clkPeriod);
177     x_in = 8'sd0;   w_in      = 8'sd0;
178     #(clkPeriod);
179     x_in = 8'sd0;   w_in      = 8'sd0;
180     #(clkPeriod);
181     x_in = 8'sd0;   w_in      = 8'sd0;
182     #(clkPeriod);
183     x_in = 8'sd0;   w_in      = 8'sd0;
184     #(clkPeriod);
185     x_in = 8'sd0;   w_in      = 8'sd0;
186     #(clkPeriod);
187     x_in = 8'sd0;   w_in      = 8'sd0;
188     wait(done); data_valid <= 0;
189
190
191
192     #(clkPeriod*2);
193     data_valid = 0; start = 0;
194
195     #(clkPeriod);
196     $stop;
197
198 end
199
200 endmodule

```

The following section summarizes the simulation test plan and verification outcomes for the perceptron_top hardware module. The primary objective of this testbench is to evaluate the correctness of the multiply-accumulate operation, verify proper handshaking communication (start, data_valid, done), demonstrate reset recovery, confirm accurate bias addition, and validate numerical stability at extreme signed 8-bit boundary values.

Test Environment & Design Parameters:

- **Time Scale:** 1ns / 100ps
- **Clock Period:** 10 ns
- **Input Data Width (DATA_WIDTH):** 8 bits (Signed, range: -128 to +127)
- **Accumulator Width (ACC_WIDTH):** 32 bits
- **Input Vector Size (N):** 8

Detailed Test Cases Plan:

Test Case 1: Standard Positive Inputs

- **Objective:** Verify basic functional operation with standard positive inputs and zero bias.
- **Bias:** 0
- **Input Pairs (x, w):**
 1. (5, 4)
 2. (3, 6)
 3. (2, 1)
 4. (4, 2)
 5. (1, 5)
 6. (6, 1)
 7. (2, 3)
 8. (3, 2)

Expected Mathematical Sum: 71

Test Case 2: Asynchronous Reset Recovery Test

- **Objective:** Ensure the system resets properly mid-execution/between runs and clears internal accumulators and registers.
- **Procedure:** Assert $\text{rst_n} = 0$ for 1 clock cycle, then release $\text{rst_n} = 1$.
- **Verification Focus:** Confirm state machine clears and registers reset to default baseline before starting subsequent operations.

Test Case 3: Standard Negative Inputs

- **Objective:** Verify signed arithmetic handling with negative inputs ($x_{\text{in}} < 0$) and positive weights ($w_{\text{in}} > 0$).
- **Bias:** 0
- **Input Pairs (x, w):**
 1. (-10, 3)
 2. (-5, 2)
 3. (-3, 4)
 4. (-1, 1)
 5. (-2, 5)
 6. (-4, 2)
 7. (-1, 3)
 8. (-2, 1)

Expected Mathematical Sum: -76 ($\text{ReLU}(-76) = 0$ so $\text{final_output} = 0$)

Test Case 4: Mixed Extreme Bounds Multiplication

- **Objective:** Verify corner-case performance on product boundaries ($-128 * 127$).
- **Bias:** 0
- **Input Pairs (x, w):**
 - Pair 1: (-128, 127)
 - Pairs 2–8: (0, 0)

Expected Mathematical Sum: -16,256 (final_output = 0)

Test Case 5: Full-Scale Accumulator Stress Test

- **Objective:** Verify accumulator expansion and bit-width dynamic range under continuous maximum positive multiplication across all N=8 inputs.
- **Bias:** 0
- **Input Pairs (x, w):**
 - Pairs 1–8: (127, 127)

Expected Mathematical Sum: 129,032

Test Case 6: Non-Zero Bias Addition

- **Objective:** Verify that the system correctly incorporates a non-zero, negative bias value into the final output calculation.
- **Bias:** -128
- **Input Pairs (x, w):**
 - Pair 1: (5, 2)
 - Pair 2: (5, 2)
 - Pairs 3–8: (0, 0)

Expected Mathematical Sum: $5*2 + 5*2 - 128 = -108$ (final_output = 0)

- Simulation Results:

```
# Time =          0|rst_n = 0|start = 0|data_valid = 0|x_in =    1|w_in =    1|bias =    0|done = 0|final_output =    0
# Time =        200|rst_n = 1|start = 0|data_valid = 0|x_in =    0|w_in =    0|bias =    0|done = 0|final_output =    0
# Time =        300|rst_n = 1|start = 1|data_valid = 0|x_in =    0|w_in =    0|bias =    0|done = 0|final_output =    0
# Time =        400|rst_n = 1|start = 1|data_valid = 1|x_in =    5|w_in =    4|bias =    0|done = 0|final_output =    0
# Time =        450|rst_n = 1|start = 1|data_valid = 1|x_in =    5|w_in =    4|bias =    0|done = 0|final_output =    20
# Time =        500|rst_n = 1|start = 0|data_valid = 1|x_in =    3|w_in =    6|bias =    0|done = 0|final_output =    20
# Time =        550|rst_n = 1|start = 0|data_valid = 1|x_in =    3|w_in =    6|bias =    0|done = 0|final_output =    38
# Time =        600|rst_n = 1|start = 0|data_valid = 1|x_in =    2|w_in =    1|bias =    0|done = 0|final_output =    38
# Time =        650|rst_n = 1|start = 0|data_valid = 1|x_in =    2|w_in =    1|bias =    0|done = 0|final_output =    40
# Time =        700|rst_n = 1|start = 0|data_valid = 1|x_in =    4|w_in =    2|bias =    0|done = 0|final_output =    40
# Time =        750|rst_n = 1|start = 0|data_valid = 1|x_in =    4|w_in =    2|bias =    0|done = 0|final_output =    48
# Time =        800|rst_n = 1|start = 0|data_valid = 1|x_in =    1|w_in =    5|bias =    0|done = 0|final_output =    48
# Time =        850|rst_n = 1|start = 0|data_valid = 1|x_in =    1|w_in =    5|bias =    0|done = 0|final_output =    53
# Time =        900|rst_n = 1|start = 0|data_valid = 1|x_in =    6|w_in =    1|bias =    0|done = 0|final_output =    53
# Time =        950|rst_n = 1|start = 0|data_valid = 1|x_in =    6|w_in =    1|bias =    0|done = 0|final_output =    59
# Time =       1000|rst_n = 1|start = 0|data_valid = 1|x_in =    2|w_in =    3|bias =    0|done = 0|final_output =    59
# Time =       1050|rst_n = 1|start = 0|data_valid = 1|x_in =    2|w_in =    3|bias =    0|done = 0|final_output =    65
# Time =       1100|rst_n = 1|start = 0|data_valid = 1|x_in =    3|w_in =    2|bias =    0|done = 0|final_output =    65
# Time =       1150|rst_n = 1|start = 0|data_valid = 1|x_in =    3|w_in =    2|bias =    0|done = 0|final_output =    71
# [t=1350 ps] Internal Raw Sum = 71 | final output = 71
```

```

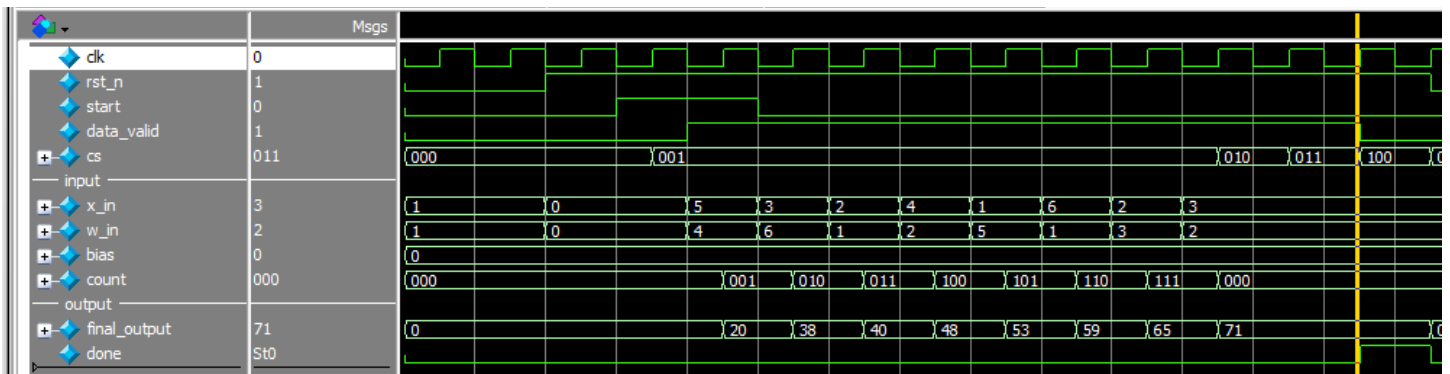
# Time = 1350|rst_n = 1|start = 0|data_valid = 0|x_in = 3|w_in = 2|bias = 0|done = 1|final_output = 71
# Time = 1450|rst_n = 0|start = 0|data_valid = 0|x_in = 3|w_in = 2|bias = 0|done = 0|final_output = 0
# Time = 1550|rst_n = 1|start = 0|data_valid = 0|x_in = 3|w_in = 2|bias = 0|done = 0|final_output = 0
# Time = 1650|rst_n = 1|start = 1|data_valid = 0|x_in = 3|w_in = 2|bias = 0|done = 0|final_output = 0
# Time = 1750|rst_n = 1|start = 1|data_valid = 1|x_in = -10|w_in = 3|bias = 0|done = 0|final_output = 0
# Time = 1850|rst_n = 1|start = 0|data_valid = 1|x_in = -5|w_in = 2|bias = 0|done = 0|final_output = 0
# Time = 1950|rst_n = 1|start = 0|data_valid = 1|x_in = -3|w_in = 4|bias = 0|done = 0|final_output = 0
# Time = 2050|rst_n = 1|start = 0|data_valid = 1|x_in = -1|w_in = 1|bias = 0|done = 0|final_output = 0
# Time = 2150|rst_n = 1|start = 0|data_valid = 1|x_in = -2|w_in = 5|bias = 0|done = 0|final_output = 0
# Time = 2250|rst_n = 1|start = 0|data_valid = 1|x_in = -4|w_in = 2|bias = 0|done = 0|final_output = 0
# Time = 2350|rst_n = 1|start = 0|data_valid = 1|x_in = -1|w_in = 3|bias = 0|done = 0|final_output = 0
# Time = 2450|rst_n = 1|start = 0|data_valid = 1|x_in = -2|w_in = 1|bias = 0|done = 0|final_output = 0
# [t=2750 ps] Internal Raw Sum = -48 | final_output = 0
# Time = 2750|rst_n = 1|start = 0|data_valid = 0|x_in = -2|w_in = 1|bias = 0|done = 1|final_output = 0
# Time = 2850|rst_n = 1|start = 1|data_valid = 0|x_in = -2|w_in = 1|bias = 0|done = 0|final_output = 0
# Time = 2950|rst_n = 1|start = 1|data_valid = 1|x_in = -128|w_in = -128|bias = 0|done = 0|final_output = 0
# Time = 3050|rst_n = 1|start = 0|data_valid = 1|x_in = 0|w_in = 0|bias = 0|done = 0|final_output = 0
# [t=3950 ps] Internal Raw Sum = 0 | final_output = 0
# Time = 3950|rst_n = 1|start = 0|data_valid = 0|x_in = 0|w_in = 0|bias = 0|done = 1|final_output = 0
# Time = 4050|rst_n = 1|start = 1|data_valid = 0|x_in = 0|w_in = 0|bias = 0|done = 0|final_output = 0
# Time = 4150|rst_n = 1|start = 1|data_valid = 1|x_in = -128|w_in = 127|bias = 0|done = 0|final_output = 0
# Time = 4250|rst_n = 1|start = 0|data_valid = 1|x_in = 0|w_in = 0|bias = 0|done = 0|final_output = 0
# [t=5150 ps] Internal Raw Sum = 0 | final_output = 0
# Time = 5150|rst_n = 1|start = 0|data_valid = 0|x_in = 0|w_in = 0|bias = 0|done = 1|final_output = 0
# Time = 5250|rst_n = 1|start = 0|data_valid = 0|x_in = 0|w_in = 0|bias = 0|done = 0|final_output = 0
# Time = 5350|rst_n = 1|start = 1|data_valid = 0|x_in = 0|w_in = 0|bias = 0|done = 0|final_output = 0
# Time = 5450|rst_n = 1|start = 1|data_valid = 1|x_in = 127|w_in = 127|bias = 0|done = 0|final_output = 0
# Time = 5550|rst_n = 1|start = 0|data_valid = 1|x_in = 127|w_in = 127|bias = 0|done = 0|final_output = 16129
# Time = 5650|rst_n = 1|start = 0|data_valid = 1|x_in = 127|w_in = 127|bias = 0|done = 0|final_output = 32258
# Time = 5750|rst_n = 1|start = 0|data_valid = 1|x_in = 127|w_in = 127|bias = 0|done = 0|final_output = 48387
# Time = 5850|rst_n = 1|start = 0|data_valid = 1|x_in = 127|w_in = 127|bias = 0|done = 0|final_output = 64516
# Time = 5950|rst_n = 1|start = 0|data_valid = 1|x_in = 127|w_in = 127|bias = 0|done = 0|final_output = 80645
# Time = 6050|rst_n = 1|start = 0|data_valid = 1|x_in = 127|w_in = 127|bias = 0|done = 0|final_output = 96774
# Time = 6150|rst_n = 1|start = 0|data_valid = 1|x_in = 127|w_in = 127|bias = 0|done = 0|final_output = 112903
# Time = 6250|rst_n = 1|start = 0|data_valid = 1|x_in = 127|w_in = 127|bias = 0|done = 0|final_output = 129032
# [t=6450 ps] Internal Raw Sum = 129032 | final_output = 129032

# Time = 6450|rst_n = 1|start = 0|data_valid = 0|x_in = 127|w_in = 127|bias = 0|done = 1|final_output = 129032
# Time = 6550|rst_n = 1|start = 0|data_valid = 0|x_in = 127|w_in = 127|bias = -128|done = 0|final_output = 128904
# Time = 6650|rst_n = 1|start = 1|data_valid = 0|x_in = 127|w_in = 127|bias = -128|done = 0|final_output = 0
# Time = 6750|rst_n = 1|start = 1|data_valid = 1|x_in = 5|w_in = 2|bias = -128|done = 0|final_output = 0
# Time = 6850|rst_n = 1|start = 0|data_valid = 1|x_in = 5|w_in = 2|bias = -128|done = 0|final_output = 0
# Time = 6950|rst_n = 1|start = 0|data_valid = 1|x_in = 0|w_in = 0|bias = -128|done = 0|final_output = 0
# [t=7750 ps] Internal Raw Sum = 10 | final_output = 0
# Time = 7750|rst_n = 1|start = 0|data_valid = 0|x_in = 0|w_in = 0|bias = -128|done = 1|final_output = 0
# Time = 7850|rst_n = 1|start = 0|data_valid = 0|x_in = 0|w_in = 0|bias = -128|done = 0|final_output = 0
** Note: $stop : M:/NTII Digital Design Using FPGA/Final Project/topmodule/tb_perceptron_top.v(196)
# Time: 805 ns Iteration: 0 Instance: /tb_perceptron_top
# Break in Module tb_perceptron_top at M:/NTII Digital Design Using FPGA/Final Project/topmodule/tb_perceptron_top.v line 196

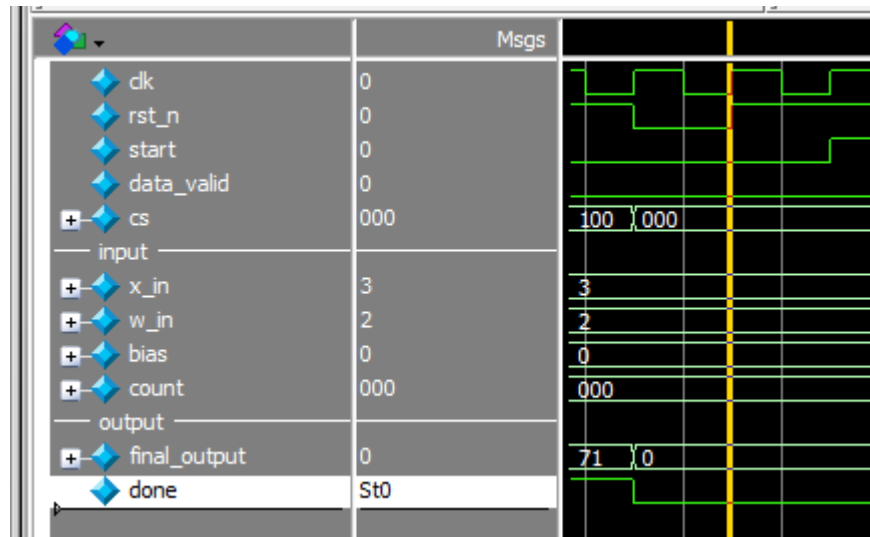
```

10.Waveform:

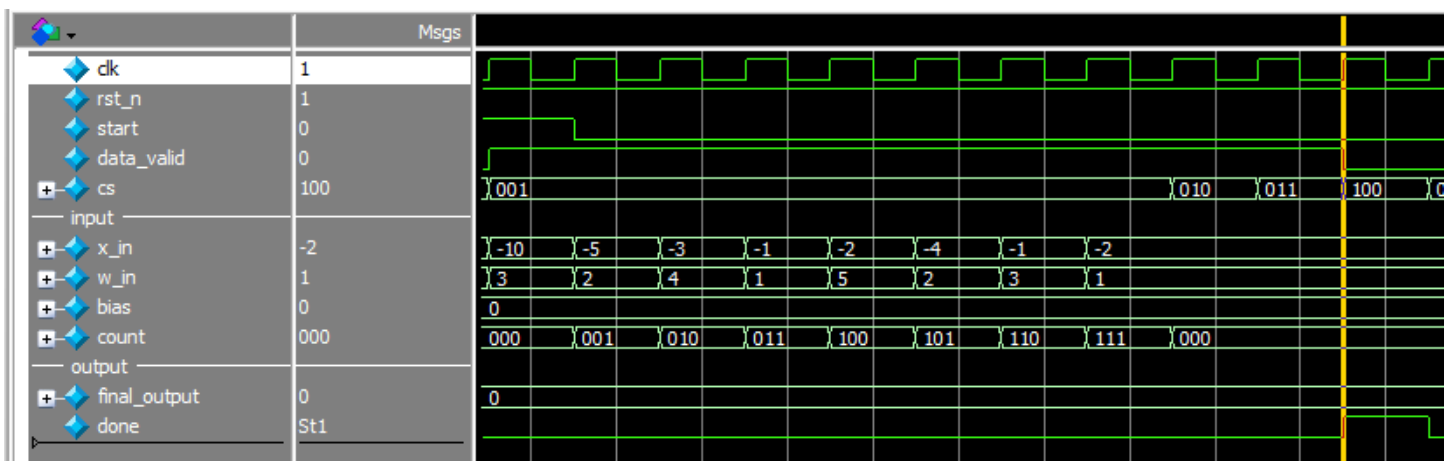
- Test case 1



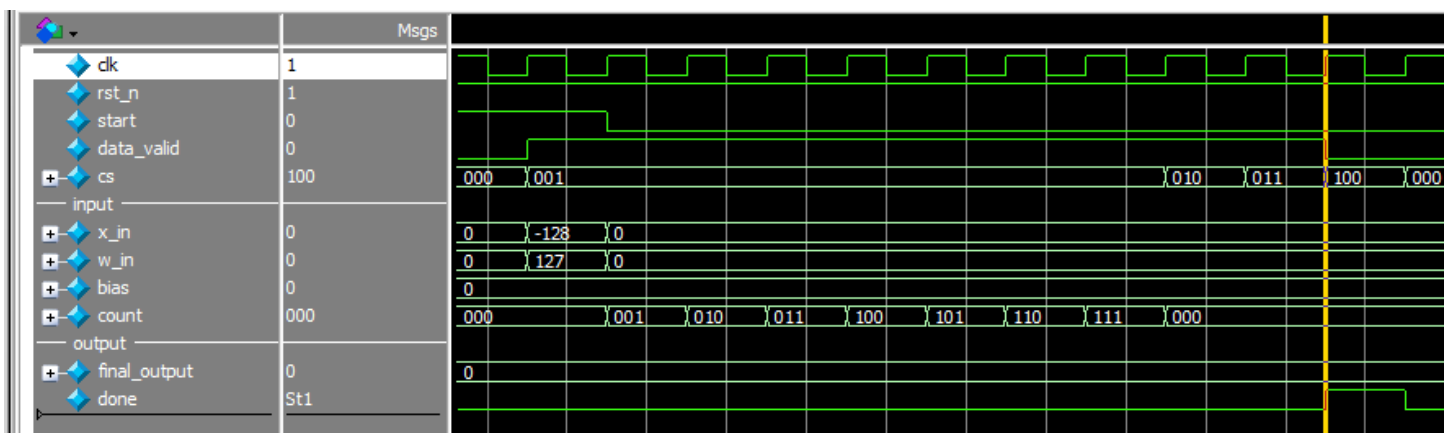
- Test case 2



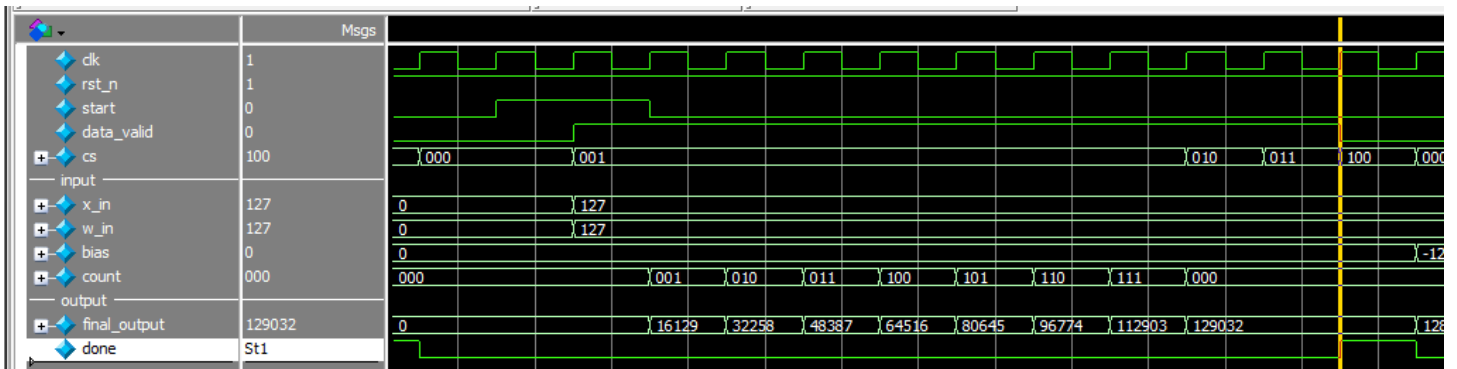
- Test case 3



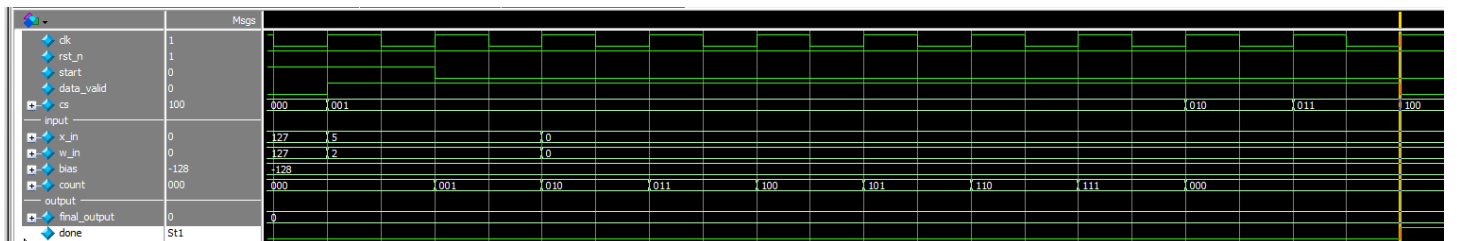
- Test case 4



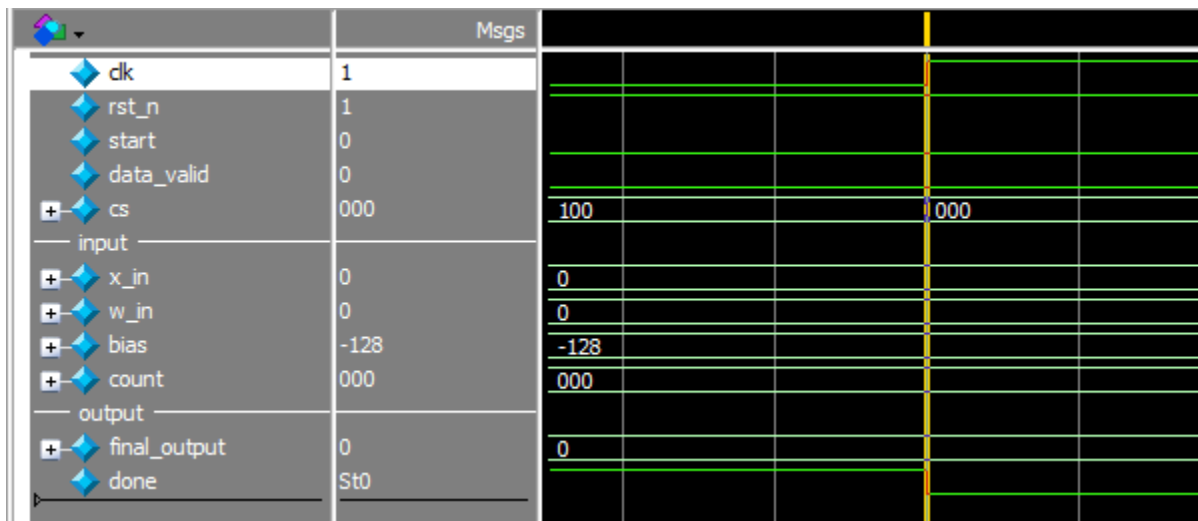
- Test case 5



- Test case 6

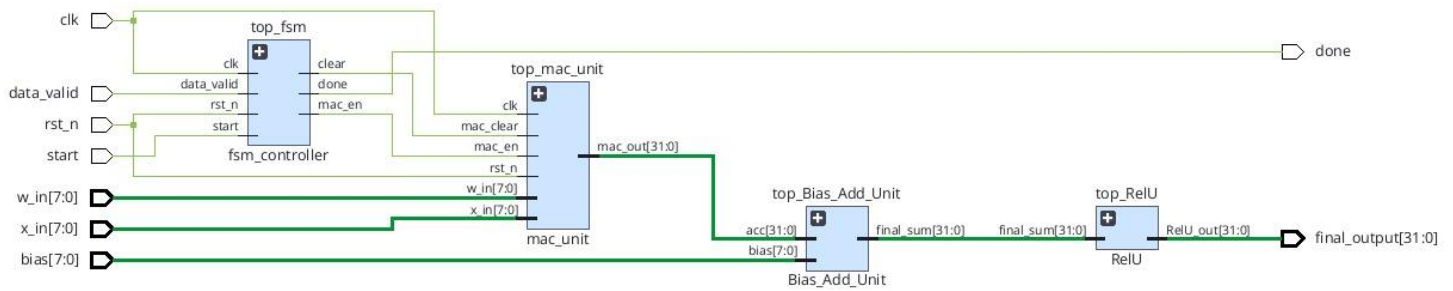


- After the ending of all test cases, start = 0 and data_valid = 0, it returns to state IDLE



11. Schematics:

- Perceptron schematic



- Layer schematic

